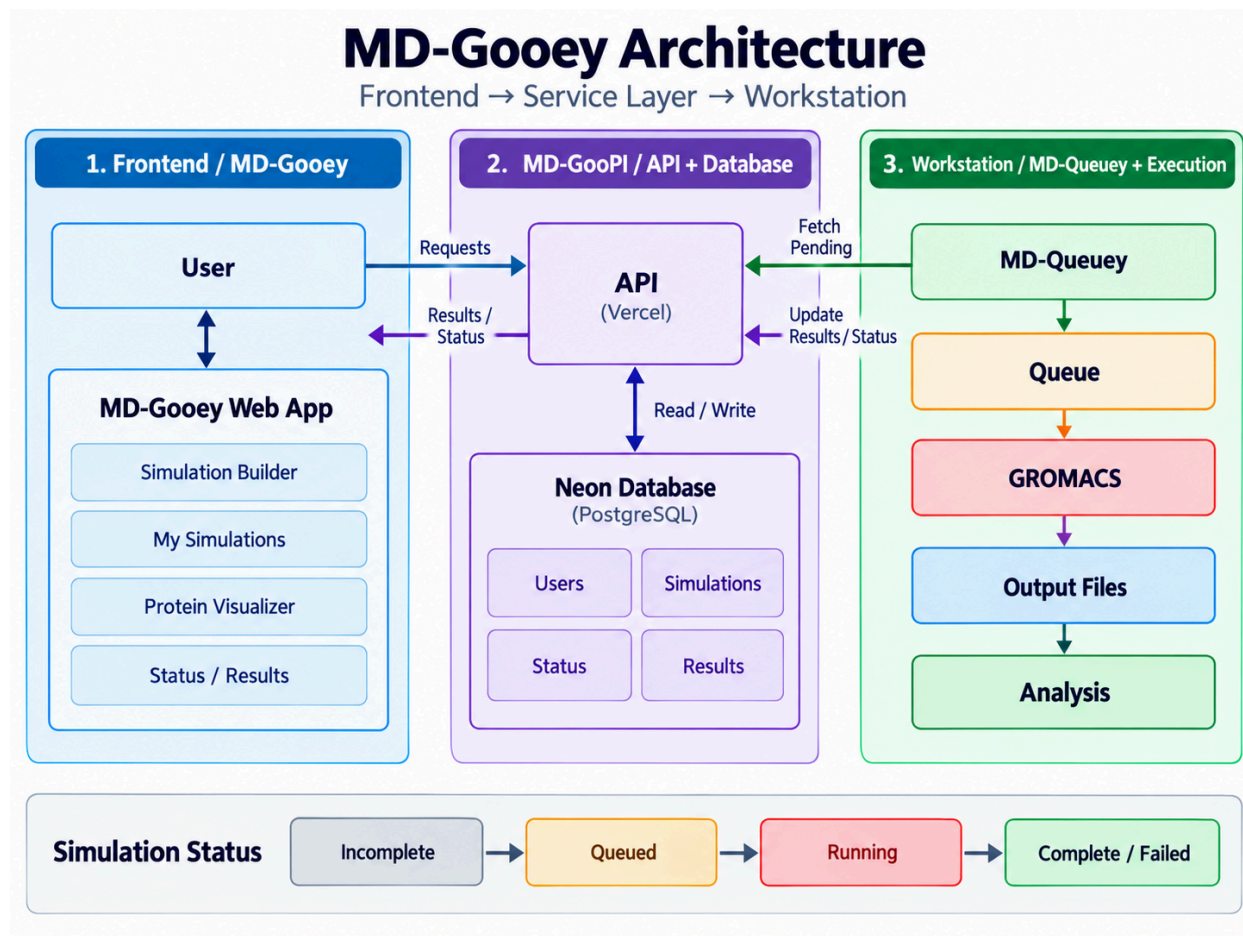


5. Software Requirements Specification (SRS)

5.1 Requirements Introduction

Diagram of iteration:



MD-Gooey is a web-based simulation management platform designed to simplify how Dr. Alavi's Molecular Physics Dynamics Research prepares, submits, and retrieves molecular dynamics simulations. The system lets users configure GROMACS simulations through an expanded graphical web interface instead of manual command-line parameters, and sends those jobs from authenticated users to a secure, multi-GPU workstation for execution. Completed simulation outputs will be stored and returned to users in a database wrapped by an API for both the daemon and frontend to use, these make up MD-GooPI, the service layer of our product. This will replace the current workflow that spans multiple devices and platforms. Separate from this database/workstation flow, protein molecule visualization functionality will be added to the frontend.

MD-Gooey, MD-GooPI, and MD-Queuey are developed using an Agile methodology. As such, system requirements are expressed primarily as user stories, which capture functional requirements from the perspective of the end user and emphasize delivered value. These user stories serve as the contractual specification between the stakeholders, Dr. Alavi, the course instructor, and the development team. They are intended to be refined and decomposed throughout the project lifecycle as stakeholder feedback is gathered and features are expanded or reduced to fit time and customer expectations.

At a high level, the system consists of the following components:

- A client-facing web application for building simulation parameters, submitting requests, monitoring job status, and viewing results.
- A persistent data store and service layer that holds user accounts, simulation requests, and simulation outputs, and authenticates all communication with the local workstation.
- A local staging application on the multi-GPU workstation that retrieves authenticated simulation requests, runs them in GROMACS, and returns the outputs to the data store. The workstation strictly does not access the general internet.
- A protein 3d visualizer will be added, which will allow the user to see the visual structure of any protein molecule they want to simulate.

The remainder of this document is structured as follows. Section 5.2 describes the CSCI, section 5.3 describes the functional requirements of the system using Agile user stories, section 5.4 describes the performance requirements of the system, and section 5.5 describes the environment requirements for both development and execution of the system.

5.2 Computer Software Configuration Items

5.2.1 MD-GooPI (the database layer) CSC

The database will sit between the pre-existing frontend and the workstation, and will store queued simulations. To save a simulation on the frontend, a user must be authenticated, and so this database will store authenticated users as well.

5.2.1.1 Neon Database

This is the database itself, created and hosted with Neon, it will contain a table of simulations, and a table of users. Simulations will have a field containing user_id, so as to be mapped to users and displayed when a particular user is authenticated on the frontend.

5.2.1.2 API Layer CSU

This is an API built with Express.js and hosted on Vercel that will allow interaction with the database from the frontend- add simulation, edit simulation, delete simulation, fetch simulations.

5.2.2 MD-Queuey CSC

This is a daemon that will run on the workstation to fetch and execute simulations from the database above.

5.2.2.1 The Script CSU

This is the program that will actually run locally on the workstation, it will fetch and queue simulations from the database with an API, will turn simulations (fetched in the form of JSON objects containing simulation parameters) into bash scripts, will then run these scripts, and will send the outputs of these simulations back to the Neon database.

5.2.2.2 API Layer CSU

This is the same API that faces the frontend, it will have a separate route for the daemon to use that will allow the fetching of all incomplete simulations.

5.2.3 MD-Gooey Extension CSC

To support the functionality introduced by MD-GooPI, the user interface of the pre-existing web app must be updated, and authentication must be added.

5.2.3.1 My Simulations Page CSU

This is a page accessible to an authenticated user where they can view and interact with their created simulations, and see simulation status and visualized results.

5.2.3.2 Authentication Page CSU

This page will allow users to create an account with their lion account, or email/password, or sign in.

5.2.3.3 Save Simulation Button CSU

A button added to the last screen of the pre-existing simulation creation flow that allows authenticated users to save their simulation, and prompts unauthenticated users to create an account or sign in.

5.2.3.4 “Currently Authenticated” UX CSU

Visual queues around the UI that notify the user that they are authenticated- will include a “Hello <username>”-type message at the top of the screen. In addition, a sign-in button will be visible if unauthenticated, a sign out button if authenticated. If authenticated, the user will also be able to see a “My Simulations” button on the navbar that will route to their simulations page.

5.2.3.4 Protein Visualizer CSU

A page allowing users to view a 3d model of a protein.

5.3 Functional Requirements

5.3.1 MD-GooPI

5.3.1.1 Shall allow the fetching of simulations created by an authenticated user, by the same authenticated user, on the frontend.

5.3.1.2 Shall allow the editing, addition, and deletion of simulations by an authenticated user.

- 5.3.1.3 Shall allow the fetching of incomplete simulations.
- 5.3.1.4 Shall allow the marking of simulations as complete.
- 5.3.1.5 Shall store authenticated users.
- 5.3.1.6 Shall associate each simulation with its creating user.
- 5.3.1.7 Shall provide simulation status information (e.g., incomplete, queued, running, complete, failed).
- 5.3.1.7 Shall store simulation results.

5.3.2 MD-Queuey

- 5.3.2.1 Shall fetch incomplete simulations from the database using MD-GooPI.
- 5.3.2.2 Shall queue incomplete simulations.
- 5.3.2.3 Shall format simulations into bash scripts.
- 5.3.2.4 Shall execute the bash scripts it creates.
- 5.3.2.5 Shall store the output files of the bash scripts it creates locally on the workstation.
- 5.3.2.6 Shall use MD-GooPI to update a simulation's status in the database.
- 5.3.2.7 Shall update the simulation status when execution begins.
- 5.3.2.8 Shall handle simulations that fail during execution.
- 5.3.2.9 Shall run continuously in the background on the workstation.
- 5.3.2.10 Shall continuously check for incomplete simulations.
- 5.3.2.11 Shall upload a simplified version of the simulation results into the Neon database.

5.3.3 MD-Gooey Extension

- 5.3.3.1 Shall provide a My Simulations page displaying simulations created by the currently authenticated user.
- 5.3.3.2 Shall allow users to create an account using their Lion account or email and password.
- 5.3.3.3 Shall allow users to authenticate using their registered account credentials.
- 5.3.3.4 Shall allow authenticated users to sign out of their account.
- 5.3.3.5 Shall allow authenticated users to save a completed simulation from the simulation creation flow.
- 5.3.3.6 Shall prompt unauthenticated users to authenticate before saving a simulation.
- 5.3.3.7 Shall display each user's saved simulations with results visualization (if they are ready) and current status.
- 5.3.3.8 Shall allow authenticated users to select a saved simulation to view its details.
- 5.3.3.9 Shall allow authenticated users to edit saved simulations.
- 5.3.3.10 Shall allow authenticated users to delete saved simulations.
- 5.3.3.11 Shall display the authenticated user's username in the application interface.
- 5.3.3.12 Shall display a sign-in option when no user is authenticated and a sign-out option when a user is authenticated.
- 5.3.3.13 Shall provide navigation to the My Simulations page when a user is authenticated.
- 5.3.3.14 Shall provide a page that visualizes protein molecules from the protein molecule database.

5.4 Performance Requirements

This section defines how well MD-Gooey must perform. Simulation run time depends on GROMACS and the molecular system, so it is not a requirement here. Where exact needs are still unknown, targets will be set with Dr. Alavi as the project develops.

5.4.1 Simulation Requests Confirmed Within Seconds

After a user submits a request, the web app shall confirm it was saved to the Neon database within 5 seconds. This includes cases where the database is waking from an idle state.

5.4.2 New Requests Reach the LMU Machine Within Minutes

The staging device shall retrieve new requests through the Vercel-hosted API and pass them to the LMU machine's queue within 10 minutes of submission. Each API call shall finish within Vercel's function time limit.

5.4.3 Users Alerted of Simulation Status Changes Within 5 Minutes

When a simulation is queued, starts, finishes, or fails, the user shall see the new status in the web app and receive email confirmation within 5 minutes.

5.4.4 Queue Runs Simulations Across All Available GPUs

The queue shall run up to four simulations at once, one on each of the machine's available GPUs. It shall also assign several GPUs to a single simulation when requested. Pending simulations shall wait in the queue without being lost.

5.4.5 Interrupted Simulations Resume After a Restart

After a power loss or reboot, the LMU machine shall restore its queue. Interrupted simulations shall resume from their last GROMACS checkpoint.

5.4.6 MD-Gooey Leaves All Possible Memory to GROMACS

MD-Gooey's background software on the LMU machine shall use only a small share of the machine's CPU and memory. It shall leave all possible GPU's free for GROMACS.

5.4.7 Storage Use Tracked Against Available Space

The LMU machine shall track how much storage simulation outputs use. It shall warn administrators before free space on the filesystem falls below a set threshold.

5.4.8 Visualizer Stays Smooth for Typical Protein Systems

The visualizer shall rotate and zoom typical protein systems without visible lag.

5.4.9 Protein Structures Load Offline in Seconds

The LMU machine shall load protein structures from its local copy of the Protein Data Bank within a few seconds.

5.4.11 Web App Available Throughout the Semester

The web app, Vercel API, and Neon database shall be available whenever Dr. Alavi's team needs them, apart from planned maintenance.

5.5 Project Environment Requirements

5.5.1 Development Environment Requirements

5.5.1.1 Access to the MD-Gooey GitHub repository.

5.5.1.2 VS Code or another compatible IDE.

5.5.1.3 JavaScript, TypeScript, Python, runtimes installed on development machines.

5.5.1.4 A personal computer with reliable internet access.

5.5.2 Execution Environment Requirements

5.5.2.1 User permissions of Dr. Alavi to allow MD-Queuey access to the workstation's hard drive.

5.5.2.2 An API authorization token stored as a GitHub secret so that the frontend has access to MD-GooPI.

5.5.2.3 A database pool URL so that MD-GooPI has access to the Neon database.

5.5.2.4 GROMACS installed and configured on the workstation.

5.5.2.5 Sufficient disk space on the workstation to store simulation input and output files.

5.5.2.6 Network access from the workstation to the MD-GooPI API.

5.5.2.7 An authorization token stored on the workstation to access MD-GooPI.

5.5.2.8 Access to the MD-Gooey GitHub repository for deployment and updates.